

Hướng dẫn sử dụng

Đường đi đầy đủ, từ lần đăng nhập đầu tiên tới lúc kết quả test quay về ticket.
Tài liệu này mô tả **hai** ứng dụng: **EmeHub** — nơi khai báo danh tính, credential và project — và **Q-Agent** — nơi công việc QA thực sự diễn ra. D-Agent chạy trong cùng suite nhưng chưa tiêu thụ cấu hình từ hub, nên nó không nằm trong luồng dưới đây (ROADMAP.md, KNOWN-GAPS.md).
Những bước còn phải làm thủ công đều được ghi rõ là thủ công. Không có bước nào bị giấu đi.

0. Trước khi bắt đầu

Ba ứng dụng nằm sau **một địa chỉ duy nhất**, agent gắn theo path. Bắt đầu từ EmeHub.

Ứng dụng	Đường dẫn
EmeHub	https://hub.chuongnd.click
Q-Agent	https://hub.chuongnd.click/qagent/
D-Agent	https://hub.chuongnd.click/dagent

Tài khoản demo nằm ở ACCOUNT.md.
Đăng nhập một lần ở EmeHub là dùng được cả ba. Thanh địa chỉ không đổi khi chuyển ứng dụng.
Cần chuẩn bị trước, không có thì tắc:

- Một **Personal Access Token** của Azure DevOps (hoặc Jira / GitHub) — để đồng bộ ticket.
- Một file **.credentials.json** của Claude CLI đang đăng nhập — để chạy các bước AI. Nếu admin đã publish credential dùng chung thì bỏ qua được bước này (xem §2).
- Node.js** trên máy của bạn, nếu muốn chạy Playwright bằng Local Agent (xem §8).

Phần A — EmeHub

1. Đăng nhập và làm quen giao diện

Vào https://hub.chuongnd.click, bấm vào để tới màn đăng nhập, nhập email và mật khẩu. Nếu tài khoản đã bật 2FA, nhập thêm mã TOTP.
Sau khi vào, sidebar bên trái chia làm hai nhóm:

WORKSPACE

Mục	Nó là gì
Overview	Trung tâm điều khiển — tình trạng từng agent, số liệu tổng, nút Launch sang agent
All projects	Danh sách project, và bảng so sánh giữa các project
Unassigned	Work item chưa thuộc project nào — chúng vẫn phải xuất hiện ở đâu đó

PLATFORM

Mục	Nó là gì
Claude Settings	Credential Claude và lựa chọn model

Authentication	Phiên đăng nhập và phương thức đăng nhập
User Management	Thành viên của workspace
Integrations	Kết nối Azure DevOps, Jira, GitHub
Settings	Giao diện và bật/tắt sản phẩm

Cây project nằm ngay dưới **All projects** với số đếm cập nhật theo dữ liệu thật.

Góc trên bên phải là **chip credential Claude** — nó cho biết bạn đang chạy bằng credential cá nhân hay credential dùng chung, còn bao nhiêu hạn mức phiên và hạn mức tuần. Bấm vào chip mở popover, từ đó nhảy thẳng sang Claude Settings.

Ở góc dưới trái là chip người dùng; bấm vào để tới **Your account** (thông tin cá nhân, đổi mật khẩu, bật 2FA).

***Mẹo:** command palette mở bằng phím tắt và nhảy được tới mọi trang, cộng thêm hai hành động nhanh — Invite member và New project.*

2. Claude Settings — credential cho AI

Đây là bước dễ bị bỏ quên nhất, và không có nó thì mọi bước AI phía sau đều không chạy. Vào **Claude Settings**. Màn hình có hai tab: **Credentials** và **Models**.

2.1. Lấy file credential

Trên máy bạn, đăng nhập Claude CLI. File `.credentials.json` do CLI tạo ra chính là thứ cần upload.

2.2. Chọn nguồn credential

Hub phân giải theo đúng thứ tự **own → shared → none**:

- **Personal** — credential của riêng bạn. Kéo thả file `.credentials.json` vào vùng upload trong mục credential cá nhân.
- **Shared** — credential do admin publish ở cấp workspace, trong mục **SHARED CLAUDE ACCOUNTS**. Chỉ admin thấy vùng upload này. Thành viên thường chỉ dùng nó.
- **Không có** — hub nói thẳng là không có credential nào được gắn, thay vì im lặng thất bại.

Công tắc **Shared | Personal** đổi qua lại tức thì. Đổi xong, chip trên header đổi theo ngay.

***Vì sao có hai lớp:** người mới vào đội chạy được từ phút đầu bằng credential chung, còn ai có tài khoản Claude riêng thì dùng của mình và tự thấy chi tiêu của mình. Cả đội không phải xếp hàng sau một tài khoản duy nhất.*

2.3. Đọc trạng thái cho đúng

Trạng thái credential **không** suy ra từ mỗi cái mốc hết hạn:

Trạng thái	Nghĩa là
active	Đang dùng được
expiring	Sắp hết hạn
refreshable	Access token đã quá hạn nhưng có refresh token — Claude CLI tự làm mới ở lần chạy tới. Đây là trạng thái bình thường, không phải lỗi.
expired	Claude CLI thật sự bị từ chối. Đây mới là tín hiệu phải upload lại.

Access token của Claude OAuth chỉ sống vài giờ, nên một file thật gần như quá hạn ngay khi upload. Khi CLI tự gia hạn, agent ghi token mới ngược về hub, nên hub luôn giữ bản mới nhất.

2.4. Kiểm tra và theo dõi

- Nút **test** gọi thử một lượt để xác nhận credential thật sự chạy được.
- Tab **Models** chọn model mặc định.
- Mỗi lượt gọi Claude được ghi lại token và chi phí; số liệu cộng dồn hiện trên chip credential.

Hub không bao giờ trả credential về trình duyệt. Endpoint chỉ trả metadata; đúng một hàm trong toàn bộ backend trả ra chất liệu credential, và nó chỉ phục vụ agent (SELLING-POINTS.md §1).

3. Integrations — kết nối provider

Vào **Integrations**. Đây là nơi khai báo Azure DevOps, Jira hoặc GitHub.

Mỗi kết nối cần:

- Loại provider** — Azure DevOps / Jira / GitHub.
- Org hoặc base URL** — ví dụ `https://dev.azure.com/<tổ-chức>`.
- Personal Access Token** — mã hoá at-rest ngay khi lưu.

Một kết nối tự khai **năng lực** của nó:

- `work_item` — cung cấp được ticket;
- `repository` — cung cấp được repo.

Nên một project gán được **hai provider khác nhau cho hai việc khác nhau** — ví dụ lấy ticket từ Azure DevOps nhưng lấy repo từ GitHub.

***PAT không bao giờ rời khỏi hub.** Hub tự proxy mọi lời gọi provider; agent không cầm PAT. Sau khi lưu, giao diện chỉ báo đã có token, không bao giờ hiện lại chính token đó — kể cả với admin. Muốn đổi thì nhập token mới.*

Sau khi lưu, dùng chức năng kiểm tra kết nối để xác nhận PAT còn hiệu lực trước khi đi tiếp.

4. Tạo project và repository

Vào **All projects** → tạo project mới (hoặc dùng lệnh *New project* trong command palette).

Mở một project ra, nó có sáu tab, mỗi tab là một địa chỉ riêng

(`/app/projects/<id>/<tab>`):

Tab	Làm gì ở đây
Overview	Tình trạng tổng quan của project
Project knowledge	Dựng và xem Knowledge Base
Repository	Danh sách repo của project; một repo được đánh dấu mặc định
Agents	Agent nào đang gán với project này
Tickets	Work item của project, provider suy ra từ chính project
Settings	Base URL, URL theo môi trường, test account, và các key/value thêm

4.1. Repository

Một project thường có nhiều repo. Thêm chúng ở tab **Repository** — lấy từ provider có năng lực repository, hoặc nhập tay. **Đánh dấu một repo là mặc định**: đó là ứng dụng được nhắm tới khi một lượt chạy không chỉ định repo nào.

4.2. Settings — phần quyết định chất lượng output

Cấu hình ở đây đi thẳng vào prompt của các bước AI phía sau. Điền đủ thì spec sinh ra chạy được gần như ngay; điền thiếu thì spec đầy chỗ trống phải sửa tay.

- **Base URL** và **URL theo môi trường** — app thật nằm ở đâu.
- **Test account** — tài khoản để spec đăng nhập. Mật khẩu mã hoá at-rest và che trong giao diện.
- **Key/value thêm** — bất cứ hằng số nào spec cần.

5. Project knowledge — dựng nền cho AI

Vào tab **Project knowledge** và chạy dựng Knowledge Base.

Việc này chạy **trên hub**, không phải trên agent — đây là artefact duy nhất hub tự dựng, vì hub đã sở hữu toàn bộ đầu vào của nó (ADR 0007).

Quá trình: hub materialise credential Claude vào một thư mục khoá chặt, chạy skill project-bootstrap qua Claude CLI, và skill này **đọc mã nguồn thật của repo** để rút ra:

- stack và kiến trúc;
- domain và các route thật;
- selector thật;
- luồng đăng nhập / xác thực;
- các môi trường;
- Page Object và fixture tái dùng được.

Kết quả lưu thành knowledge.md + knowledge.json, **theo từng repository**.

***Chờ được:** một lượt dựng đầy đủ tốn khoảng 20 phút. Dựng một lần, mọi bước AI sau đó đọc lại thay vì đọc lại code từ đầu. Chỉ dựng lại khi repo đổi đáng kể.*

Rút ngắn bằng clone. Nếu admin đã dựng sẵn một project mẫu trong namespace dùng chung, thành viên **clone** nó về scope của mình — kèm cấu hình, test account đã mã hoá và toàn bộ artefact trên đĩa — thay vì mỗi người chạy lại 20 phút.

6. Ticket

Tab **Tickets** của project hiện work item đồng bộ từ provider. Provider **suy ra từ chính project**, không phải một ô chọn — một project một nguồn ticket.

- Đồng bộ để kéo work item mới về.
- Mở một ticket ra để xem mô tả, acceptance criteria, comment, attachment, label, độ ưu tiên, trạng thái và PR liên quan.
- Ticket ở hub là **chỉ đọc**. Nơi sửa nó là provider.

Unassigned ở sidebar chứa work item chưa thuộc project nào. Nó tồn tại để không dòng nào biến mất khỏi giao diện chỉ vì chưa được gán project.

7. Quản trị workspace

User Management — thành viên workspace. Hub có đúng hai vai: admin và member. Mời người mới **tạo tài khoản ngay lập tức**, không có hàng đợi lời mời chờ duyệt.

Authentication — phiên đăng nhập và phương thức đăng nhập. Mỗi phiên là một lần đăng nhập trên một thiết bị, kèm user agent và IP.

Thu hồi một phiên là đăng xuất thiết bị đó khỏi mọi agent, vì mỗi access token đều mang theo id của phiên. Không phải thu hồi ba lần ở ba nơi.

Settings — giao diện (bốn tông màu nhấn, chế độ sáng/tối) và **bật/tắt từng sản phẩm**. Tắt một sản phẩm thì cả app, edge proxy và **trang landing** đều chặn nó — đọc lỗi trạng thái nghĩa là đóng, không phải mở.

Your account — thông tin cá nhân, đổi mật khẩu, bật 2FA (TOTP).

Mọi việc đáng ghi đều vào **audit log** append-only: ai làm, làm gì, lên đối tượng nào, từ IP nào.

8. Sang Q-Agent

Từ **Overview**, bấm **Launch** ở card Q-Agent.

Không có màn đăng nhập thứ hai. Hub mint một access token gắn đúng audience của Q-Agent; Q-Agent tự xác thực token đó cục bộ.

Phần B — Q-Agent

Q-Agent là nơi công việc QA diễn ra: từ ticket ra test case, ra spec Playwright, chạy thật, thu bằng chứng, rồi trả kết quả về ticket.

9. Project trong Q-Agent

Vào **Projects**, mở project ra. Các tab:

Tab	Nội dung
Overview	Tổng quan project
Tickets	Work item
Runs	Các lượt chạy
Knowledge	Knowledge Base theo repo
Connection	Kết nối provider của project
Reports	Báo cáo

Màn **Getting started** dẫn qua các bước thiết lập nếu đây là lần đầu.

10. Run — đơn vị công việc trung tâm

Run là thực thể trung tâm. Mọi thứ sau bước chọn ticket đều thuộc về một Run.

Tạo run từ tab **Tickets**: chọn một ticket, một nhóm ticket đã chọn, toàn bộ ticket được gán cho mình, hoặc cả sprint.

Một Run đi qua **tám chặng**, và mỗi chặng là một địa chỉ riêng (/projects/<guid>/runs/<runId>/<chặng>). Thanh **PipelineRail** hiện chặng hiện tại trên mọi màn hình thuộc run.

processing → review → sync → automation → executing → evidence → comment → done

10.1. processing — AI phân tích và sinh test case

Không cần thao tác. Hai skill chạy nối tiếp:

- requirement-analyst đọc ticket + Knowledge Base, ra bản phân tích yêu cầu;
- test-case-generator biến bản phân tích đó thành test case thủ công kiểu Azure DevOps.

Chủ ý ở chặng này là ít, không phải nhiều. test-case-generator chỉ sinh happy path — luồng nghiệp vụ chính, mỗi acceptance criterion một kịch bản thành công. Nó cố tình không sinh case biên, case âm, ma trận quyền hay bộ regression. Đó là việc của test-case-reviewer ở chặng sau. Một bộ nhỏ để review hơn một bộ trăm case không ai đọc.

10.2. review — Review Center: cửa duyệt của con người

Đây là cửa quan trọng nhất trong toàn bộ luồng. Test case sinh ra ở trạng thái pending.

Ở Review Center, bạn:

- đọc và sửa từng test case;
- **approve** hoặc **reject**;
- chỉnh **automation type** của từng case: Playwright / Selenium / Cypress / Manual;
- chạy thêm một lượt AI review (test-case-reviewer) để mở rộng độ phủ — case biên, case âm, validation, quyền, rủi ro regression.

Chỉ case approved mới đi tiếp. Case Manual không bao giờ bị đem đi sinh spec.

10.3. sync — Create & Link

Tạo các test case đã duyệt lên provider và link chúng với work item gốc.

Có chế độ dry-run (local mode): ghi lại phía Q-Agent, không viết gì lên provider. Dùng nó khi đang thử sản phẩm — không ai phải chọn giữa "demo" và "làm bản project thật của khách hàng".

10.4. automation — sinh spec Playwright

Đây là chặng khác biệt nhất về kỹ thuật. Có **hai chế độ authoring**, chọn được:

live-harness — live authoring (khuyến nghị). Một Claude tác tử lái **trình duyệt thật đã đăng nhập** qua CLI browser-harness: thực hiện từng bước của test case trên app thật, tìm phần tử qua accessibility tree, ghi lại **selector thật** theo thứ tự ưu tiên data-testid → ARIA role

- tên → label → CSS ổn định, tự tạo dữ liệu test nếu chưa có, xác nhận từng kết quả mong đợi —

rồi mới emit spec, dựng từ đúng những gì đã chạy được.

Điểm mấu chốt: nó xác minh **selector**, không phải toạ độ. Một cú click theo toạ độ trúng vào bất kỳ phần tử nào nằm ở điểm đó, trong khi spec sẽ click chính selector đã ghi. Nên mỗi

thao tác được dispatch thẳng trên selector sắp đưa vào spec và kiểm tra hiệu ứng thật. Kết quả là spec chạy xanh ngay, không cần lượt heal nào.

blind — sinh từ Knowledge Base rồi tự chữa. Claude viết spec từ KB + Project Config, chạy, và khi hỏng thì đưa lỗi kèm DOM sống ngược lại để sửa có mục tiêu — tối đa 3 lượt, timeout rút ngắn, chạy trên model rẻ. Một lượt heal thành công có bám DOM sẽ ghi selector **verified-at-runtime** ngược vào Knowledge Base, nên lần sau đã khôn hơn.

Cả hai chế độ đều bị chặn bằng trần chi phí, trần số lượt và timeout theo đồng hồ.

Cửa chặn trước khi chạy — placeholder gate. Spec sinh ra bị soi trước khi được phép thực thi: selector bịa, stub TODO và URL giữ chỗ đều bị chặn. Verdict là passed / blocked / rejected. Spec blocked hiện lên như *cần grounding thêm* — thường là dựng lại KB hoặc chạy một lượt exploration — thay vì lặng lẽ fail lúc chạy và bị đọc nhầm thành lỗi sản phẩm.

10.5. executing — chạy thật

Playwright chạy thật bộ spec đã duyệt. Mỗi case có trạng thái riêng: pending / running / pass / fail.

Chọn nơi chạy: **Local Agent** (mặc định) hoặc server. Xem §11.

Mỗi case fail được execution-analyzer phân loại nguyên nhân:

Phân loại	Nghĩa là
test_defect	Spec sai
product_defect	Ứng dụng sai — đây mới là bug thật
flaky	Không ổn định
environment	Lỗi môi trường
timeout	Quá thời gian

Việc này giữ cho một spec hỏng không bị đọc thành một bug sản phẩm.

10.6. evidence — bằng chứng

Với mỗi case đã chạy: ảnh chụp màn hình, video, Playwright trace, console log, network log và tóm tắt. Nhóm theo ticket.

Ảnh chụp màn hình **chú thích được** — khung, mũi tên, highlight, vòng tròn, chữ — để đính kèm vào comment gửi về ticket.

10.7. comment — trả kết quả về ticket

Comment kết quả được **chuẩn bị và cho xem trước**, không phải ghi thẳng. Bạn xem, sửa nếu cần, rồi mới publish lên work item gốc. Mapping trạng thái cấu hình được (ví dụ *Ready for QA* → *Testing* → *Passed* / *QA Failed*).

10.8. done

Báo cáo tổng hợp: kết quả chung, tóm tắt theo ticket, số pass/fail, phân tích lỗi bằng AI, thời gian, môi trường và link tới evidence. Xem lại bất cứ lúc nào ở tab **Reports**.

11. Local Agent — chạy test trên máy của bạn

Vì sao cần: ứng dụng nằm sau SSO/MFA cần một con người đăng nhập thật trong một trình duyệt có giao diện. Local Agent chạy spec ngay trên máy tester, nên **cookie và**

storageState không bao giờ rời khỏi thiết bị — chỉ spec, kết quả và evidence đi ngược về server.

Nó cũng là nơi thực hiện self-heal phía agent, DOM exploration, live authoring và capture đăng nhập thủ công.

Ghép đôi thiết bị (bắt buộc, làm một lần)

- 1. Vào màn **Local agent** trong Q-Agent, lấy **mã ghép đôi** ngắn hạn.
- 2. Trên máy bạn, chạy Local Agent: `npx @q-agent/agent` (hoặc bản desktop Electron — **hiện chỉ có installer cho Windows**).
- 3. Nhập mã. CLI đổi nó lấy một token lâu dài của riêng thiết bị, lưu tại `~/ .qagent-agent/config.json`. Server chỉ giữ bản hash.
- 4. Từ đó, mọi job đều gắn với chủ sở hữu thiết bị. Admin thu hồi token bất cứ lúc nào.

Đăng nhập thủ công một lần

Với app sau SSO/MFA: Local Agent mở trình duyệt thật, **bạn tự đăng nhập bằng tay**, và phiên đó được giữ lại tại chỗ cho các lượt chạy sau. Quy tắc cứng: agent **không bao giờ** tự gõ mật khẩu hay mã MFA, và **không bao giờ** chạy trên môi trường production.

12. Cài đặt trong Q-Agent

Màn hình	Nội dung
Settings	Cài đặt chung
Settings › Users	Quản lý người dùng
Settings › Claude credentials	Credential Claude phía Q-Agent
Settings › Shared workspace	Namespace dùng chung — admin dựng project mẫu ở đây, thành viên clone về
Audit log	Nhật ký append-only
Profile	Tài khoản cá nhân

Ngôn ngữ giao diện đổi được giữa tiếng Anh và tiếng Việt. Đây thuần túy là tùy chọn hiển thị lưu trong `localStorage`, đổi có hiệu lực tức thì, **không** ảnh hưởng tới dữ liệu, payload gửi provider hay các artefact sinh ra — những thứ đó theo ngôn ngữ của ticket gốc.

13. Khi có gì đó không chạy

Triệu chứng	Nguyên nhân thường gặp
Bước AI không chạy	Chưa có credential Claude, hoặc credential ở trạng thái expired. Vào Claude Settings, đổi sang Shared hoặc upload lại file mới.
Credential hiện màu cảnh báo nhưng vẫn chạy	Trạng thái refreshable — bình thường. Access token quá hạn nhưng refresh token còn; CLI tự làm mới. Không cần làm gì.
Đồng bộ ticket không ra gì	PAT hết hạn hoặc thiếu scope. Kiểm tra lại kết nối ở Integrations.
Test case sinh ra chung chung, đầy chỗ trống	Chưa dựng Knowledge Base, hoặc Project settings thiếu base URL / test account. Cả hai đều đi thẳng vào prompt.

Spec bị blocked	Placeholder gate bắt được selector chưa có căn cứ. Dừng lại Knowledge Base hoặc chạy live authoring thay vì blind.
Không chạy được execution	Local Agent chưa ghép đôi, hoặc chưa chạy trên máy. Xem §11.
Chạy trên app sau SSO thì hỏng ở bước đăng nhập	Cần một lần đăng nhập thủ công trên Local Agent. Agent không tự gõ mật khẩu.
Đăng xuất một thiết bị	Authentication trong EmeHub — thu hồi phiên là đăng xuất khỏi cả ba ứng dụng.

Đọc tiếp

- ACCOUNT.md — link truy cập và tài khoản demo
- SELLING-POINTS.md — vì sao suite này làm khác
- KNOWN-GAPS.md — những gì đã gỡ và những gì chưa làm
- CONTEXT.md — từ vựng dùng chung
- INTEGRATION.md — hợp đồng giữa hub và agent